

Viste e Viste Materializzate

Esercizio 6 – Creare una vista logica per reporting

Crea una vista chiamata report_dipartimenti che mostri: Nome del dipartimento, Numero di corsi associati, Numero totale di studenti iscritti a tali corsi. Obiettivo: utilizzare aggregazioni e JOIN in una vista.

```
CREATE VIEW report_dipartimenti AS
SELECT
    d.nome AS nome_dipartimento,
    COUNT(DISTINCT c.id_corso) AS numero_corsi,
    COUNT(DISTINCT i.id_studente) AS numero_studenti
FROM dipartimenti d
JOIN docenti do ON d.id_dipartimento = do.id_dipartimento
JOIN corsi c ON do.id_docente = c.id_docente
JOIN iscrizioni i ON c.id_corso = i.id_corso
GROUP BY d.id_dipartimento, d.nome;
```

Esercizio 7 – Viste annidate

Crea una vista report_docenti_corsi che mostri per ogni docente: Nome e cognome, Numero di corsi tenuti, Media dei voti dei propri studenti. Poi crea una seconda vista report_docenti_eccellenti che mostri solo i docenti con media voti superiore a 27. Obiettivo: comprendere la composizione di viste e riutilizzo logico.

```
CREATE VIEW report_docenti_corsi AS
SELECT
    d.nome, d.cognome,
    COUNT(DISTINCT c.id_corso) AS corsi_tenuti,
    AVG(e.voto) AS media_voti
FROM docenti d
LEFT JOIN corsi c ON d.id_docente = c.id_docente
LEFT JOIN esami e ON e.id_corso = c.id_corso
GROUP BY d.id_docente, d.nome, d.cognome;
```

```
CREATE VIEW report_docenti_eccellenti AS
SELECT nome, cognome, corsi_tenuti, media_voti
```

```
FROM report_docenti_corsi  
WHERE media_voti > 27;
```

Esercizio 8 – Simulare una vista materializzata

Crea una tabella report_studenti_media contenente: ID studente, Nome completo, Media voti, Fascia di rendimento (CASE WHEN). Popola la tabella con una query aggregata da studenti ed esami. Poi crea una procedura che aggiorni la tabella (TRUNCATE + INSERT) per mantenerla allineata. Obiettivo: simulare una vista materializzata e comprenderne il ciclo di aggiornamento.

```
CREATE TABLE report_studenti_media (  
    id_studente INT PRIMARY KEY,  
    nome_completo VARCHAR(100),  
    media_voti DECIMAL(10,2),  
    fascia_rendimento VARCHAR(20)  
);
```

```
DELIMITER //
```

```
CREATE PROCEDURE aggiorna_report_studenti_media()  
BEGIN  
    TRUNCATE TABLE report_studenti_media;  
    INSERT INTO report_studenti_media (id_studente, nome_completo, media_voti, fascia_rendimento)  
    SELECT  
        s.id_studente,  
        CONCAT(s.nome, ' ', s.cognome),  
        media_voti_studente(s.id_studente),  
        fascia_rendimento(media_voti_studente(s.id_studente))  
    FROM Studenti s;  
END //  
DELIMITER ;
```

Esercizio 9 – Analisi di performance

Confronta i tempi di esecuzione tra: SELECT s.nome, AVG(e.voto) FROM studenti s JOIN esami e ON s.id_studente = e.id_studente GROUP BY s.id_studente; e SELECT * FROM report_studenti_media;

Obiettivo: dimostrare i benefici delle viste materializzate su query complesse.

Query con JOIN: Richiede il calcolo in tempo reale di medie e collegamenti tra studenti ed esami, analizzando 90 righe con l'uso di tabelle temporanee (Using temporary).

Query su Vista Materializzata: Esegue una semplice lettura (type: ALL) sulla tabella report_studenti_media che contiene già i risultati pronti.

Il caso della tabella report_studenti_media è più veloce perché evita il ricalcolo dei dati a ogni esecuzione.

Esercizio 10 – Reporting finale

Crea una vista report_facolta che mostri: Nome della facoltà, Numero di dipartimenti, Numero medio di studenti per corso, Media generale dei voti degli studenti della facoltà. Obiettivo: costruire una vista di reporting avanzato multilivello, integrando più tabelle.

```
CREATE VIEW report_facolta AS
SELECT
    f.nome AS nome_facolta,
    COUNT(DISTINCT d.id_dipartimento) AS numero_dipartimenti,
    AVG(studenti_per_corso.num_studenti) AS media_studenti_per_corso,
    AVG(e.voto) AS media_voti_facolta
FROM facolta f
LEFT JOIN dipartimenti d ON f.id_facolta = d.id_facolta
LEFT JOIN docenti doc ON d.id_dipartimento = doc.id_dipartimento
LEFT JOIN corsi c ON doc.id_docente = c.id_docente
LEFT JOIN (
    SELECT id_corso, COUNT(DISTINCT id_studente) AS num_studenti
    FROM iscrizioni GROUP BY id_corso
) studenti_per_corso ON studenti_per_corso.id_corso = c.id_corso
LEFT JOIN esami e ON e.id_corso = c.id_corso
GROUP BY f.id_facolta, f.nome;
```